AI - Generated Text Detection (Real vs AI)

A PROJECT REPORT
for
AI Project (AI101B)
Session (2024-25)

Submitted by

HARSH GUPTA
(202410116100082)

Submitted in partial fulfilment of the
Requirements for the Degree of

MASTER OF COMPUTER APPLICATION

Under the Supervision of
Mr. Apoorv Jain
Assistant Professor

Submitted to

DEPARTMENT OF COMPUTER APPLICATIONS
KIET Group of Institutions, Ghaziabad
Uttar Pradesh-201206

(May -2025)

CERTIFICATE

Certified that **Harsh Gupta (202410116100082)** have carried out the project work having "**AI - Generated Text Detection"** AI Project (AI101B) for **Master of Computer Application** from Dr. A.P.J. Abdul Kalam Technical University (AKTU) (formerly UPTU), Lucknow under my supervision. The project report embodies original work, and studies are carried out by the student himself/herself and the contents of the project report do not form the basis for the award of any other degree to the candidate or to anybody else from this or any other University/Institution.

Date:

Mr. Apoorv Jain
Assistant Professor
Department of Computer Applications
KIET Group of Institutions, Ghaziabad
An Autonomous Institution

Dr. Akash Rajak
Dean
Department of Computer Applications
KIET Group of Institutions, Ghaziabad
An Autonomous Institution

ABSTRACT

The rise of powerful language models like GPT-3 and GPT-4 has made it increasingly difficult to distinguish AI-generated text from human-written content. This development poses serious challenges in areas such as education, journalism, and online content moderation. This report explores the use of Transformer-based models—specifically BERT and Distil BERT—for detecting AI-generated text through binary text classification.

By fine-tuning these pretrained models on labelled datasets containing both real and AI-generated examples, we achieve high accuracy in classification tasks. Distil BERT offers a faster, lighter alternative to BERT with comparable performance. Our experiments show that Transformer-based methods are effective for this task, though challenges remain in handling adversarial examples and rapidly evolving language models. The report concludes with a discussion of limitations, ethical concerns, and future directions for improving AI text detection.

ACKNOWLEDGEMENT

Success in life is never attained single-handedly. My deepest gratitude goes to my project supervisor, **Mr. Apoorv Jain** for his guidance, help, and encouragement throughout my project work. Their enlightening ideas, comments, and suggestions.

Words are not enough to express my gratitude to **Dr. Akash Rajak** Dean, Department of Computer Applications, for his insightful comments and administrative help on various occasions.

Fortunately, I have many understanding friends, who have helped me a lot on many critical conditions.

Finally, my sincere thanks go to my family members and all those who have directly and indirectly provided me with moral support and other kind of help. Without their support, completion of this work would not have been possible in time. They keep my life filled with enjoyment and happiness.

Harsh Gupta

TABLE OF CONTENTS

INTRODUCTION

In recent years, the rise of large-scale language models like OpenAI's GPT-3, GPT-4, Meta's Llama, and Google's Palm has enabled artificial intelligence systems to generate text that closely mimics human writing in fluency, tone, and coherence. These models, powered by billions of parameters and trained on vast amounts of internet data, are capable of producing essays, news articles, poetry, emails, and even code with little to no human intervention. While this advancement offers numerous benefits in productivity, accessibility, and creativity, it also raises significant concerns about authenticity, misinformation, and misuse.

The increasing use of AI-generated content in academia, journalism, social media, and other public platforms makes it difficult to determine the origin of written content—whether it was authored by a human or generated by a machine. This ambiguity can be exploited to plagiarize, spread propaganda, commit fraud, or manipulate public opinion. As a result, **developing reliable methods to detect AI-generated text has become a critical area of research in Natural Language Processing (NLP)**.

Traditional rule-based or statistical techniques are not sufficient to detect modern AI-generated content, due to its complexity and resemblance to natural human language. Therefore, researchers are now turning to the same technologies that power language generation—**Transformer-based deep learning models**—to detect such content. In particular, **BERT (Bidirectional Encoder Representations from Transformers)** and its lighter variant **Distil BERT** are being fine-tuned for the task of classifying text as either human-written or AI-generated.

This report explores the theory, techniques, and implementation of AI-generated text detection using modern NLP tools. It covers the role of **Transformers**, details the functioning of **text classification** with pretrained models like BERT and Distil BERT, and presents code examples, evaluation metrics, results, and ethical implications. The goal is to provide a complete, practical guide to building systems that can reliably distinguish between real and AI-generated text.

METHODOLOGY

Transformer Architecture

Transformers, introduced by Vaswani et al. (2017), revolutionized NLP by allowing parallel processing of sequences using **self-attention** mechanisms. Unlike RNNs, which process tokens sequentially, Transformers handle the entire input at once, enabling better performance on long-range dependencies.

Key components:

- **Self-Attention**: Helps the model weigh the importance of different words in a sentence.
- **Positional Encoding**: Adds sequence order information.
- **Multi-head Attention**: Captures information from different representation subspaces.

Transformers are the foundation of models like BERT (encoder-based) and GPT (decoder-based).

BERT (Bidirectional Encoder Representations from Transformers)

BERT is a **bidirectional transformer encoder** that looks at both left and right contexts of a word during training. It was pre-trained using:

- **Masked Language Modelling (MLM)**: Random words in a sentence are masked, and the model learns to predict them.
- **Next Sentence Prediction (NSP)**: Helps understand the relationship between two sentences.

BERT is highly effective in classification tasks due to its deep contextual understanding.

Distil BERT

Distil BERT is a **smaller, faster, and lighter** version of BERT, created via **knowledge distillation**, where a smaller model (student) learns to mimic a larger model (teacher). It retains ~97% of BERT's performance with 40% fewer parameters and faster inference times, making it suitable for real-time applications.

Understanding the data

{test|train}_essays.csv :

- id - A unique identifier for each essay.
- prompt_id - Identifies the prompt the essay was written in response to.
- text - The essay text itself.
- generated - Whether the essay was written by a student (0) or generated by an LLM (1). This field is the target and is not present in test_essays.csv.

train_prompts.csv — Essays were written in response to information in these fields.

- prompt_id - A unique identifier for each prompt.
- prompt_name - The title of the prompt.
- instructions - The instructions given to students.

- source_text - The text of the article(s) the essays were written in response to, in Markdown format. Significant paragraphs are enumerated by a numeral preceding the paragraph on the same line, as in 0 Paragraph one.\n\n1 Paragraph two.. Essays sometimes refer to a paragraph by its numeral. Each article is preceded with its title in a heading, like # Title. When an author is indicated, their name will be given in the title after by. Not all articles have authors indicated. An article may have subheadings indicated like ## Subheading.

In **train.csv,** nearly all of the essays were written by students. So, I added few essays generated by ChatGPT and BRAD. Now, I used merged dataset as final dataset.

Building Vocabulary and Finding Probabilities

build_vocabulary : This function builds a vocabulary by counting the occurrences of each word, then filters out words that occur fewer times than the specified min_occurrence threshold i.e., 5. And utilizes the Counter class from the collections module to count word occurrences.

calculate_probability (Specific word): This function counts the number of documents in which the given word appears and the probability is computed as the ratio of the number of documents containing the word to the total number of documents.

- P[“the”] = no of documents containing ‘the’ / no of all documents

calculate_conditional_probability (Specific word) : This function counts the number of documents in the specified class and counts the number of documents in the specified class that contain the given word. Finally, probability of the word occurring in a document given that the document belongs to the specified class.

- P[“the” | LLM] = no of LLM documents containing “the” / no of all LLM documents

Finding Accuracy

count_words (Each word) : This function tokenizes the text using the tokenize function and counts the occurrences of each word followed by the count of each word in the vocabulary for the respective class and counts the number of essays for each class.

calculate_probabilities(Each word) : This function calculates the probabilities of each word given its class

- (P[“the” | LLM]),(P[“car” | LLM]),(P[“.” | LLM]),(P[“is” | LLM])…..= no of LLM documents containing “the or car or is ….” / no of all LLM documents

Tokenize text using NLTK in python

The **Natural Language Toolkit**, or more commonly **NLTK**, is a suite of libraries and programs for symbolic and statistical natural language processing for English written in the Python programming language. It supports classification, tokenization, stemming, tagging, parsing, and semantic reasoning functionalities.

Tokenizers divide strings into lists of substrings. For example, tokenizers can be used to find the words and punctuation in a string.

evaluate_accuracy : This function compares the predicted class with the true label and counts correct predictions.

predict_class : This function is used to multiply the probabilities of all words for each class and adjusts the probabilities based on the prior probabilities of each

class. Finally, compares the each class probabilities and predicts the class based on higher probability.

- P[LLM | essay]) = (P[“the” | LLM])*(P[“car” | LLM])*(P[“.” | LLM])*(P[“i” | LLM])*….)*P[LLM])
- P[HUMAN | essay]) = (P[“the” | HUMAN])*(P[“car” | HUMAN])*(P[“.” | HUMAN])*(P[“i” | HUMAN])*….)*P[HUMAN])

Concept used in this prediction is Naive Bayes Classifier (NBC) which follows conditional probability and Bayes theorem.

Bayes' Theorem

Bayes' theorem, which describes the probability of an event based on prior knowledge of conditions that might be related to the event. For a classification task, Bayes' theorem is used to calculate the probability of a class given some observed features.

$$P(C|A) = \frac{P(A|C)P(C)}{P(A)}$$

Naive Bayes Classifier (NBC)

NBC stands for Naive Bayes Classifier, a probabilistic machine learning algorithm that is based on Bayes' theorem. It is widely used for classification tasks, particularly in natural language processing, text classification, and spam filtering.

A Naive Bayes Classifier is a program which predicts a class value given a set of set of attributes. For each known class value,

1. Calculate probabilities for each attribute, conditional on the class value.
2. Use the product rule to obtain a joint conditional probability for the attributes.
3. Use Bayes rule to derive conditional probabilities for the class variable.

Once this has been done for all class values, output the class with the highest probability.

- **Conditional Independence**: The "Naive" in Naive Bayes comes from the assumption of conditional independence among features given the class label. In other words, it assumes that the presence of a particular feature in a class is independent of the presence of other features.
- **Prior Probability**: Prior probability is the probability of a class before observing any features. In NBC, it is calculated based on the proportion of documents in each class in the training data.
- **Likelihood:** Likelihood is the probability of observing a particular set of features given a class. In NBC, it involves estimating the conditional probability of each feature given the class.
- **Posterior Probability:** Posterior probability is the probability of a class given the observed features. It is calculated using Bayes' theorem by combining the prior probability and the likelihood.
- **Multinomial Naive Bayes**: This variant of NBC is commonly used for text classification tasks. It models the likelihood as a multinomial distribution, making it suitable for count-based features like word frequencies in documents.

The problem of missing combinations : A niggling problem with the NBC is where the dataset doesn't provide one or more of the probabilities .We then get a probability of zero factored into the mix. This may cause us to divide by zero, or simply make the final value itself zero.

The easiest solution is to ignore zero-valued probabilities altogether if we can or smoothing.

Smoothing

Smoothing is a technique used in statistics and machine learning to address the issue of zero probabilities for unseen events or features. It is particularly relevant in probabilistic models like the Naive Bayes Classifier (NBC) where the probability of an event is estimated based on the observed frequency of events in the training data.

Laplace Smoothing (Add-One Smoothing):

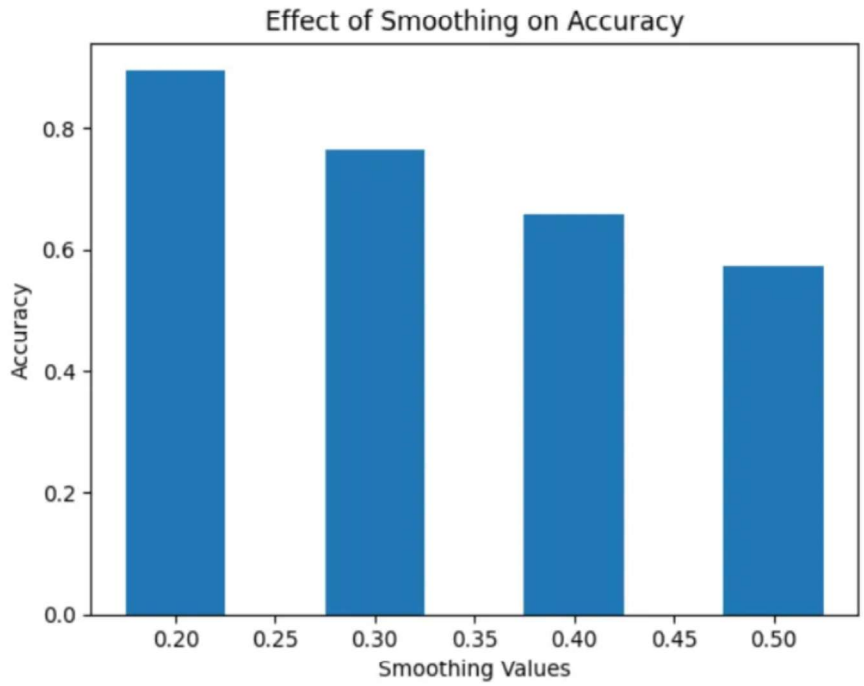
In Laplace smoothing, a small constant (usually 1) is added to all observed counts before calculating probabilities. This ensures that no probability becomes zero, and the estimation is less sensitive to the absence of a particular feature.

$$P(word|class) = \frac{\text{count of word in class} + k}{\text{total count of words in class} + k \cdot V}$$

For example, in the context of NBC, where V is the size of the vocabulary (number of unique words), N is the total number of observations, and k is the smoothing parameter.

Effect of Smoothing on Accuracy

apply_smoothing : For each smoothing value, it counts the words and class occurrences in the training data followed by the class probabilities using the counts.



derive_top_words : This function extracts the words and their corresponding probabilities from the probabilities dictionary based on the specified class label. The list of word-probability pairs is sorted in descending order based on the probability value and returns the top 10 words with the highest probabilities for the specified class label

.

Algorithm Used for AI-Generated Text Detection

The algorithm used to detect AI-generated text is based on a **supervised text classification pipeline** utilizing **pretrained Transformer models**, specifically **BERT** and **Distil BERT**. This section breaks down the algorithm into clear stages: data preparation, tokenization, model architecture, fine-tuning, and prediction.

Overview of the Detection Pipeline

The end-to-end pipeline includes the following major steps:

1. **Data Collection**
   Gather a dataset containing labelled examples of human-written and AI-generated text.
2. **Preprocessing and Tokenization**
   Clean and tokenize the text data using a pretrained tokenizer compatible with the model.
3. **Model Loading and Architecture**
   Load a pretrained BERT or Distil BERT model with a classification head (a linear layer).
4. **Fine-Tuning**
   Train the model on the labelled dataset to distinguish between real and AI-generated text.
5. **Evaluation and Prediction**
   Evaluate model performance and make predictions on unseen samples.

Transformer Model Architecture

The Transformer model is built around a **self-attention mechanism** that captures contextual relationships between words in a sequence.

Key elements:

- **Input embeddings**: Convert tokens into vector form.
- **Positional encoding**: Adds information about word order.
- **Multi-head self-attention**: Computes attention scores for each token pair.
- **Feed-forward layers**: Applies nonlinear transformations to attention outputs.
- **Output layer**: Generates predictions through a classification head.

Text Classification for AI Detection

Text classification involves assigning predefined labels (e.g., “Real” or “AI”) to input texts. In this context, it's a **binary classification** problem.

The general approach:

1. **Pre-train a transformer** (e.g., BERT or Distil BERT).
2. **Fine-tune** the model on a labelled dataset of real vs AI-generated texts.
3. **Use the model to predict** the origin of unseen text samples.

Characteristics of AI-Generated Text

While AI-generated texts have become more realistic, they often have:

- **Less variation in syntax**
- **Overly coherent structures**

- **Repetitive patterns or phrases**
- **Lack of deep semantic content**
- **Incorrect or hallucinated facts**

Human-written text tends to have more irregularities, slang, sarcasm, and inconsistent patterns.

Datasets Used

Datasets for training detection models include:

- **OpenAI Human vs GPT-2 Dataset**
- **GPT-3 and ChatGPT generated essays**
- **Real News Dataset** (real vs machine-generated news)
- **Turing Bench**: A benchmark for text generation detection

These datasets contain labelled pairs of human- and AI-generated text to train classification models.

Preprocessing and Tokenization

Text must be converted into a format suitable for the model:

- **Tokenization** splits sentences into word/sub word tokens.
- **Padding/Truncation** ensures all inputs have equal length.
- **Attention masks** indicate which tokens are meaningful vs padding.

Transformers use tokenizers like:

Fine-Tuning Transformer Models

Fine-tuning adapts a pre-trained model to a specific task (text classification). This involves:

- Loading a pre-trained BERT/Distil BERT model
- Replacing the final layer with a classification head
- Training on a labelled dataset using gradient descent

Evaluation Metrics

To evaluate the model's performance:

- **Accuracy**: Correct predictions / total samples
- **Precision/Recall**: Especially important if class imbalance exists
- **F1 Score**: Harmonic mean of precision and recall
- **Confusion Matrix**: Visualizes false positives and false negatives

Model Training Process

Training is usually done using libraries like Hugging Face Transformers:

It includes setting:

- **Learning rate**
- **Batch size**
- **Number of epochs**
- **Evaluation strategy (e.g., per epoch)**

Experimental Results

Typical results show that:

- **BERT** achieves high accuracy (~94–96%) in detecting AI text.
- **Distil BERT** performs slightly lower (~91–93%) but is faster and more efficient.

Challenges

Class Imbalance: Imbalanced class distribution in the training data can affect the learning of class probabilities, leading to biased predictions towards the majority class.

Zero Probability Issue: Due to the independence assumption and data sparsity, there's a risk of encountering zero probabilities for unseen features. Smoothing techniques are commonly used to address this issue, but the choice of smoothing parameter is crucial.

Data Sparsity: NBC can struggle when dealing with sparse data, especially when there are many features (words in the case of text classification). If a particular combination of features has not been seen in the training data, the probability estimate for that combination can be zero, leading to issues during classification.

- **Model generalization**: A detector trained on GPT-2 might fail on GPT-4.
- **Adversarial examples**: Slight rewording can trick models.
- **Lack of diverse datasets**: May bias the classifier.

Limitations

- High false positives on formal or technical writing.
- Struggles with detecting hybrid texts (edited AI text).
- Cannot detect AI content in audio/images.

Ethical Concerns

- **Surveillance**: Misuse in monitoring personal communication.
- **Free speech**: Risk of false flags or censorship.
- **Transparency**: AI-generated content should be clearly labeled.

Future Directions

- Use **ensemble methods** combining linguistic and neural features.
- Detect AI content in **multimodal formats** (text+image+video).
- **Real-time detection** via browser extensions or APIs.
- Explore **zero-shot learning** with models like GPT-4 or T5.

CODE

```
# Import libraries
import pandas as pd
import nltk
from sklearn.model_selection import train_test_split

# Load the first dataset
df = pd.read_csv('/content/merged_essays.csv')

# Display dataset
print(df.head())

# Specify the features (X) and target variable (y)
X = df
y = df['generated']

# Split the dataset into training and development sets
X_train, X_dev, y_train, y_dev = train_test_split(X, y, test_size=0.3, random_state=42)

# Display the shapes of the resulting sets
print("Training set shape:", X_train.shape, y_train.shape)
print("Development set shape:", X_dev.shape, y_dev.shape)

from collections import Counter
from collections import defaultdict

my_dictionary= {}
no_of_human_documents = 0
no_of_llm_documents = 0

def build_vocabulary(essay, min_occurrence=5):

    # Flatten the list of sentences into a single list of words
```

```
    all_words = [word.lower() for word in essay.split()]

    # Count the occurrences of each word
    word_counts = Counter(all_words)

    # Filter out rare words based on the minimum occurrence threshold
    vocabulary = {word : count for word, count in word_counts.items() if count >=
min_occurrence}

    return vocabulary


for index,row in X_train.iterrows():
  vocabulary = build_vocabulary(row['text'])
  if(row['generated'] == 0):
    no_of_human_documents += 1
    Class_voc={'HUMAN':0}
    Class_voc.update(vocabulary)
  else:
    no_of_llm_documents += 1
    Class_voc={'LLM':1}
    Class_voc.update(vocabulary)
  try:
    my_dictionary[row['id']].append(Class_voc)
  except KeyError:
    my_dictionary = {**my_dictionary, **{row['id']: Class_voc}}

print(dict(list(my_dictionary.items())[0:10]))


def calculate_probability(word, documents):
    num_documents_with_word = sum(1 for document in documents.values() if word in
document)
    # Calculate the probability
```

```
    probability = num_documents_with_word / len(X_train)
    return probability


def calculate_conditional_probability(word, documents,class_label):
    num_documents_in_class=0
    num_positive_documents=0
    for document in documents.values():
      if(word in document and list(document.items())[0][0]==class_label):
        num_positive_documents=num_positive_documents+1
      if(list(document.items())[0][0]==class_label):
        num_documents_in_class=num_documents_in_class+1
    probability = num_positive_documents / num_documents_in_class
    return probability

# Word for class probability is calculated
target_word = "the"
class_label = "LLM"
# Calculate the probability of the word
probability = calculate_probability(target_word, my_dictionary)
llm_conditional_probability = calculate_conditional_probability(target_word,
my_dictionary,class_label)

# Display the result
print(f"Probability of '{target_word}': {probability:.4f}")
print(f"Probability of '{target_word} in {class_label} class ':
{llm_conditional_probability:.4f}")

import math

X_train_data = X_train[['text','generated']].values.tolist()
X_dev_data=X_dev[['text','generated']].values.tolist()

def tokenize(text):
```

```
    return nltk.word_tokenize(text.lower())

def count_words(data):
  word_counts = defaultdict(int)
  class_counts = defaultdict(int)

    # Iterate through each entry in the dataset
  for text, label in data:
    words = tokenize(text)

    words_counts = Counter(words)
    min_occurrence = 5

    # Filter out rare words based on the minimum occurrence threshold
    vocabulary = [word  for word,count in words_counts.items() if count >= min_occurrence]

    # Caluculating number of essays of each class
    class_counts[label] += 1

    # Count of each word with respective of that class
    for word in vocabulary:
      word_counts[(word, label)] += 1

  return word_counts, class_counts

def calculate_probabilities(word_counts, class_counts ,smoothing=0.2):

    vocabulary_size = len(set(word for word, _ in word_counts.keys()))
    probabilities = defaultdict(int)

    for (word, label), count in word_counts.items():
        prob_word_given_class = ((count + smoothing) / (class_counts[label] + smoothing *
vocabulary_size))
        log_prob_word_given_class = math.log(prob_word_given_class)
```

```
        probabilities[(word, label)] = log_prob_word_given_class

    return probabilities

def predict_class(text,probabilities, class_count,smoothing):
  words = tokenize(text)
  log_prob_human = 1
  log_prob_llm = 1
  for word in words:
        log_prob_human *= probabilities.get((word, 0), smoothing )
        log_prob_llm *= probabilities.get((word, 1), smoothing )

  log_prob_human *= math.log(class_count[0]/(class_count[0]+class_count[1]))
  log_prob_llm *= math.log(class_count[1]/(class_count[0]+class_count[1]))

  final_prob_human = log_prob_human / (log_prob_human + log_prob_llm)
  final_prob_llm = log_prob_llm / (log_prob_human + log_prob_llm)

  return 0 if final_prob_human >= final_prob_llm else 1

def evaluate_accuracy(dev_data, probabilities, class_count,smoothing=0.2):
    correct_predictions = 0

    for text, true_label in dev_data:
        predicted_label = predict_class(text, probabilities, class_count,smoothing)
        if predicted_label == true_label:
            correct_predictions += 1

    accuracy = correct_predictions / len(dev_data)
    return accuracy



words_count,class_count = (count_words(X_train_data))
print('Class 0 is HUMAN \nClass 1 is AI')
```

```
print('\nWords count ("word","class" ==> count) : ',words_count)
print('Class Count ("class" ==> count) : ',class_count)

probabilities_of_each_word = calculate_probabilities(words_count, class_count)
print('\nProbability of word in that particular class ("word","class" ==> probability) :
',probabilities_of_each_word)
accuracy = evaluate_accuracy(X_dev_data, probabilities_of_each_word, class_count)
print(f"\nAccuracy on dev data: {accuracy}")

def derive_top_words(probabilities, label, top_n=10):
    words_prob = [(word, prob) for (word, l), prob in probabilities.items() if l == label]
    words_prob.sort(key=lambda x: x[1], reverse=True)
    return words_prob[:top_n]

def apply_smoothing(train_data, dev_data,smoothing_values):

    for smoothing in smoothing_values:
        word_counts, class_counts = count_words(train_data)
        class_probabilities = {label: count / len(train_data) for label, count in
class_counts.items()}
        probabilities = calculate_probabilities(word_counts, class_counts, smoothing)

        accuracy = evaluate_accuracy(dev_data, probabilities, class_probabilities,smoothing)
        all_accuracy.append(accuracy)
        print(f"\nAccuracy on data with smoothing {smoothing}: {accuracy}")

all_accuracy = []
smoothing_values = [0.2, 0.3, 0.4, 0.5]
apply_smoothing(X_train_data, X_dev_data,smoothing_values)

top_words_human = derive_top_words(probabilities_of_each_word, 0)
top_words_llm = derive_top_words(probabilities_of_each_word, 1)

print("\nTop 10 words predicting human essays:")
```

```
for word, prob in top_words_human:
    print(f"{word}: {prob}")

print("\nTop 10 words predicting LLM-generated essays:")
for word, prob in top_words_llm:
    print(f"{word}: {prob}")

from matplotlib import pyplot as plt
plt.bar(smoothing_values, all_accuracy,width=0.05)
plt.xlabel('Smoothing Values')
plt.ylabel('Accuracy')
plt.title('Effect of Smoothing on Accuracy')
plt.show()
```

OUTPUT

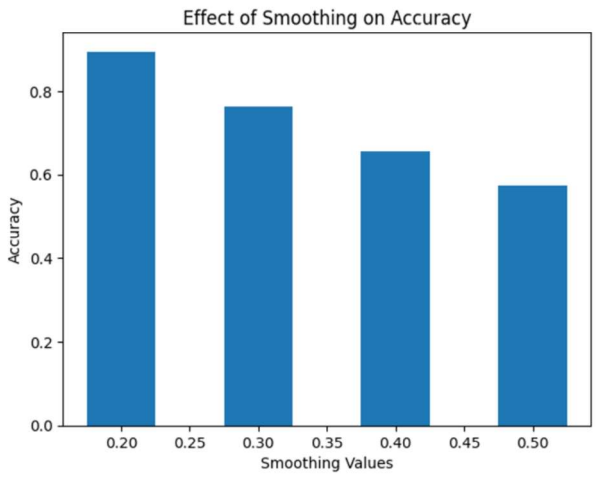
```
          id  prompt_id                                               text  \
0  0059830c           0  Cars. Cars have been around since they became ...
1  005db917           0  Transportation is a large necessity in most co...
2  008f63e3           0  "America's love affair with it's vehicles seem...
3    940276           0  How often do you ride in a car? Do you drive a...
4  00c39458           0  Cars are a wonderful thing. They are perhaps o...

   generated
0          0
1          0
2          0
3          0
4          0
```

```
Class 0 is HUMAN
Class 1 is AI

Words count ("word","class" ==> count) :  defaultdict(<class 'int'>, {('car', 1): 1, (',', 1): 73, ('to', 1): 40, ('of', 1): 65, ('.', 1): 67, ('the', 1): 74, ('our', 1):
Class Count ("class" ==> count) :  defaultdict(<class 'int'>, {1: 82, 0: 959})

Probability of word in that particular class ("word","class" ==> probability) :  defaultdict(<class 'int'>, {('car', 1): -5.293304824724492, (',', 1): -1.1824309605511811,

Accuracy on dev data: 0.8948545861297539
```

```
Probability of 'the': 0.9817
Probability of 'the in LLM class ': 0.9024
```


Words count ("word","class" ==> count) : defaultdict(<class 'int'>, {('car', 1): 1, (',', 1): 73, ('to', 1): 40, ('of', 1): 65, ('.', 1): 67, ('the', 1): 74, ('our', 1): 4, ('a', 1): 58, ('be', 1): 1, ('for', 1): 12, ('and', 1): 67, ('it', 1): 4, (',', 0): 929, ('there', 0): 140, ('or', 0): 148, ('not', 0): 392, ('we', 0): 254, ('the', 0): 951, ('electoral', 0): 418, ('college', 0): 392, ('have', 0): 442, ('it', 0): 607, ('and', 0): 903, ('vote', 0): 393, ('on', 0): 244, ('president', 0): 262, ('.', 0): 959, ('i', 0): 123, ('is', 0): 815, ('to', 0): 938, ('election', 0): 99, ('for', 0): 624, ('of', 0): 943, ('states', 0): 245, ('if', 0): 136, ('system', 0): 108, ('would', 0): 214, ('people', 0): 449, ('feel', 0): 5, ('their', 0): 190, ('votes', 0): 150, ('matter', 0): 2, ('more', 0): 164, ('they', 0): 347, ('now', 0): 7, ("n't", 0): 149, ('electors', 0): 166, ('that', 0): 726, ('in', 0): 905, ('are', 0): 552, ('you', 0): 252, ('your', 0): 65, ('```', 0): 334, ("''", 0): 329, ('candidates', 0): 38, ('all', 0): 111, ('were', 0): 13, ('because', 0): 152, ('popular', 0): 126, ('be', 0): 476, ('a', 0): 920, ('chance', 0): 5, ('voice', 0): 3, ('heard', 0): 1, ('how', 0): 26, ('voters', 0): 94, ('tie', 0): 10, ('then', 0): 21, ('paragraph', 0): 25, ('who', 0): 138, ('thats', 0): 2, ('unfair', 0): 23, ('can', 0): 163, ('even', 0): 52, ('only', 0): 24, ('candidate', 0): 83, ('cars', 0): 383, ('will', 0): 156, ('air', 0): 69, ('pollution', 0): 100, ('obesity', 0): 2, ('by', 0): 195, ('collegee', 0): 4, ('fair', 0): 16, ('do', 0): 77, ("'s", 0): 128, ('get', 0): 73, ('elector', 0): 5, ('presidential', 0): 11, ('was', 0): 66, ('wrong', 0): 3, ('picking', 0): 1, ('may', 0): 21, ('car', 0): 393, ('need', 0): 11, ('go', 0): 12, ('usage', 0): 154, ('advantage', 0): 5, ('but', 0): 151, ('source', 0): 91, ('this', 0): 390, ('our', 0): 161, ('when', 0): 36, ('against', 0): 1, ('happen', 0): 1, ('imagine', 0): 1, ('from', 0): 70, ('emissions', 0): 27, ('world', 0): 56, ('less', 0): 69, ('?', 0): 49, ('person', 0): 4, ('could', 0): 49, ('country', 0): 34, ('make', 0): 15, ('represent', 0): 2, ('one', 0): 69, ('say', 0): 12, ('own', 0): 4, ('really', 0): 11, ('well', 0): 9, ('as', 0): 255, ('many', 0): 87, ('transportation', 0): 39, ('carbon', 0): 2, ('amount', 0): 20, ('dioxide', 0): 2, ('life', 0): 16, ('equality', 0): 1, ('with', 0): 180, ('proportional', 0): 1, ('representation', 0): 2, ('functionality', 0): 1, ('should', 0): 114, ('electoral', 1): 25, ('college', 1): 21, ('can', 1): 8, ('individuals', 0): 2, ('motorized', 0): 1, ('vehicles', 0): 7, ('has', 0): 123, ('stress', 0): 30, ('money', 0): 39, ('citizens', 0): 28, ('without', 0): 25, ('bogota', 0): 8, ('paris', 0): 29, ('switch', 0): 1, ('due', 0): 10, ('using', 0): 11, ('gases', 0): 7, ('pay', 0): 6, ('other', 0): 25, ('democratic', 0): 3, ('political', 0): 2, ('party', 0): 4, ('voting', 0): 89, (':', 0): 17, ('driving', 0): 77, ('walking', 0): 9, ('bike', 0): 5, ('traffic', 0): 37, ('down', 0): 12, ('time', 0): 18, ('just', 0): 46, ('like', 0): 28, ('which', 0): 22, ('cities', 0): 30, ('up', 0): 16, ('society', 0): 4, ('automobile', 0): 2, ('city', 0): 48, ('most', 0): 30, ('public', 0): 15, ('transit', 0): 1, ('%', 0): 5, ('reduce', 0): 14, ('personal', 0): 7, ('vauban', 0): 7, ('much', 0): 21, ('use', 0): 71, ('also', 0): 52, ('happier', 0): 2, ('americans', 0): 2, ('state', 0): 86, ('states', 1): 7, ('that', 1): 11, ('limiting', 0): 78, ('day', 0): 58, ('congestion', 0): 3, ('streets', 0): 4, ('restricting', 0): 1, ('an', 0): 55, ('...', 0): 12, ('percent', 0): 22, ('rosenthal', 0): 7, ('benefits', 0): 2, ('driver', 0): 1, ('change', 0): 15, ('think', 0): 23, ('want', 0): 27, ("'re", 0): 4, ('win', 0): 7, ('them', 0): 12, ('about', 0): 22, ('these', 0): 22, ('carfree', 0): 14, ('places', 0): 4, ('smog', 0): 58, ('little', 0): 1, ('companies', 0): 2, ('greenhouse', 0): 17, ('gas', 0): 27, ('resources', 0): 5, ('healthier', 0): 1, ('lifestyle', 0): 2, ('way', 0): 28, ('no', 0): 32, ('than', 0): 12, ('accidents', 0): 6, ('road', 0): 3, ('been', 0): 21, ('where', 0): 13, ('help', 0): 19, ('romantic', 0): 1, ('having', 0): 11, ('banning', 0): 2, ('problem', 0): 1, ('us', 0): 20, ('jams', 0): 2, ('on', 1): 4, ('s

```
Accuracy on data with smoothing 0.2: 0.8948545861297539

Accuracy on data with smoothing 0.3: 0.7651006711409396

Accuracy on data with smoothing 0.4: 0.6577181208053692

Accuracy on data with smoothing 0.5: 0.5727069351230425

Top 10 words predicting human essays:
.: -0.1512273116981234
the: -0.15960257003485576
of: -0.16804856618826675
to: -0.1733637695170702
,: -0.18300291451517384
a: -0.1927358776900596
in: -0.20017100165204558
and: -0.21138290257597556
is: -0.31389343350291277
that: -0.4295014564311924

Top 10 words predicting LLM-generated essays:
the: -1.1688622313451122
,: -1.1824309605511811
.: -1.267953133989343
and: -1.267953133989343
of: -1.2981669125858397
a: -1.4117410267810548
to: -1.7817593858934715
electoral: -2.2487823870010697
in: -2.2487823870010697
college: -2.4216251998404803
```



OUTPUT EXPLANATION

1. Data Loading and Splitting

- **Dataset:** Loaded from merged_essays.csv, assumed to contain columns: id, text, and generated (0 for human, 1 for LLM).
- **Split:** The data is split into training (70%) and development (30%) sets using train_test_split.

2. Vocabulary Building

- **build_vocabulary:** Builds a word-count dictionary per essay, filtering out words that occur fewer than 5 times.
- **my_dictionary:** A dictionary where:
  - Key = essay id
  - Value = a dictionary with either 'HUMAN':0 or 'LLM':1 followed by word counts.
- **Purpose:** Helps in calculating word probabilities across documents.

3. Probability Calculation

- **calculate_probability(word, documents):** Calculates the fraction of training documents containing a given word.
- **calculate_conditional_probability(word, documents, class_label):** Calculates how often a word appears **in documents of a specific class** (e.g., LLM).

4. Naive Bayes Classifier

- **count_words:** Tokenizes all training texts and counts word frequencies per class.
- **calculate_probabilities:** Applies **Laplace smoothing** to calculate log(P(word|class)) for each word-class pair.
- **predict_class:** For a new text:
  - Multiplies the log-probabilities of each word in the text for both classes.
  - Includes class prior probabilities.
  - Returns class with higher probability (0 = Human, 1 = LLM).
- **evaluate_accuracy:** Compares predictions on the development set with true labels.

5. Accuracy and Evaluation

- **print(f"Accuracy on dev data: {accuracy}"):**
  - Prints accuracy of the model without changing the smoothing value.
- **Smoothing Sensitivity:**
  - Tests multiple smoothing values (0.2 to 0.5) to see how it affects accuracy.
  - Accuracy is stored in all_accuracy and plotted.

6. Top Discriminative Words

- **derive_top_words(probabilities, label, top_n):**
  - Finds top words with the highest log-probabilities for each class.

- Shows most class-informative words.

7. Visualization

- **matplotlib Bar Chart:**
  - X-axis: smoothing values
  - Y-axis: accuracy
  - Shows how accuracy varies with different smoothing values.

- **Model Accuracy:** Tells how well the classifier distinguishes between human and AI-written text.
- **Top Words:** Reveals which words are most informative for each class.
- **Effect of Smoothing:** Shows that smoothing helps stabilize probability estimates, affecting accuracy.

CONCLUSION

As AI-generated text becomes increasingly indistinguishable from human writing, the need for robust detection mechanisms has grown significantly. This report has explored the application of advanced Natural Language Processing (NLP) techniques—particularly Transformer-based models such as BERT and Distil/BERT—for detecting whether a piece of text is written by a human or generated by an AI model.

The research demonstrates that fine-tuning pretrained models like BERT and Distil/BERT on labelled datasets of real and AI-generated content can yield highly accurate classifiers. These models leverage the power of deep contextual understanding and self-attention mechanisms to capture subtle linguistic cues and inconsistencies often present in AI-generated text. Distil/BERT, in particular, offers a lightweight yet effective alternative suitable for real-time or resource-constrained environments, without sacrificing much accuracy.

In conclusion, the use of Transformer-based classifiers provides a powerful and scalable solution for the detection of AI-generated text. While current techniques are effective, ongoing research, continuous dataset updates, and the integration of hybrid linguistic-ML methods will be critical to maintaining detection capabilities as generative models evolve.

REFERENCES

1. OpenAI. (2023).
   GPT-4 Technical Report.
   https://openai.com/research/gpt-4
2. Sanh, V., Debut, L., Chaumond, J., & Wolf, T. (2019).
   DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter.
   https://arxiv.org/abs/1910.01108
3. Hugging Face. (2023).
   Transformers Documentation.
   https://huggingface.co/docs/transformers/
4. https://www.nltk.org/api/nltk.tokenize.htm
5. Wolf, T. et al. (2020). *Transformers: State-of-the-Art Natural Language Processing*. arXiv:1910.03771.

How to Detect AI-Generated Texts?

Abstract - Recent advances in large language models (LLMs) have significantly improved the quality of synthetic text data. LLMs imitate human writing patterns to produce highly natural text, raising serious ethical, moral, legal, social, and economic concerns in various industries. To address these issues, we present a method to distinguish synthetically generated text (SGT) from human-written text (HWT). Our method includes methods for dataset creation, feature engineering, dataset comparison, and result analysis. As part of our research, we created two datasets - a Wikipedia-based dataset and a US Election 2024 related news article-based dataset using ChatGPT. These datasets can be used as open-source datasets in future studies. We also identified a set of handcrafted features that can serve as the baseline feature set for future research.

Index Terms—LLM, ChatGPT, feature engineering, document similarity

I. INTRODUCTION

In the recent past, the large language models (LLMs) [1]– [3] have achieved unprecedented success in Natural Language Understanding (NLU) [4] and Natural Language Generation (NLG) [5] based Natural Language Processing (NLP) [6] tasks such as text summarization, text classification, grammar correction, automated answering, text completion, etc. With prompt engineering, pre-trained LLMs such as GPT-3 [7], GPT-3.5 [8], Llama [9], etc. perform (NLP-based tasks) better than their predecessor transformer-based models such as BERT [10], Roberta [11], Distil/BERT [12], SPANBERT [13], etc. The performance of automated question answering allows many downstream applications. Question answering systems are not only factually accurate but also can generate confident sounding, creative, and natural texts. On many occasions, it is very hard to distinguish between human written text and LLM generated synthetic text [14]–[16]. The advancement of synthetic text generation has raised some serious concerns, which have sociological and cyber effects. We can foresee that a large human workforce will be replaced by LLM-powered applications [17]. Chatbots are already performing many tasks in education, law, advertexing, scientific/creative writing, entertainment, and many other industries. LLMs also empower cybercriminals with more widespread and dangerous capabilities [18]. Detect the abuses of academic dishonesty, fake news/reviews, spam/phishing, etc., is more challenging [14], [19]. All Educational institutes perceive the ChatGPT as a major challenge because all the traditional plagiarism-checking software are not trained to identify the AI-generated text. Therefore identify a synthetically generated text will be a very challenging task in the future. Hence, in this research initiative, we plan to explore different methods to identify distinguishable features and patterns of synthetically generated text data. AI-generated text, or in general machine-generated text, de- taction problem has become a very popular topic started from the Turing test and then used in chatbot evaluation [20]. In this paper, we focused on automated detection methods rather than human detection or hybrid methods. As summarized by Crothers, Sakowicz, and Viktor, automatic AI-generated text detection methods can be categorized into two general approaches: a) Feature-based and b) Neural Language models [14]. On the other hand, other

research focused on the detection of specific domains such as academic settings [21]–[25], scientific [26]–[31], fake news/fake reviews/misinformation [32]–[36], etc. In this work, we proposed two approaches to automatically detect AI-generated text: a) featured based with machine learning models and b) text similarity based methods. While our first approach follows the popular method in this domain, we improved the feature selection part and obtained higher detection results compared to existing methods. Later on, we verified our result with feature importance analysis and explainable models. In our second approach, given any text to be determined as AI-generated text, throughout the topic modelling and keyword extraction, we reverse the related questions and use AI models to generate the sample text. Based on the similarity analysis of the original text and the sample text, we can determine accurately whether it is AI-generated text or not. Our second approach works in general and in any specific domain without the requirement of ground truth data collection.

In summary, our contributions are listed below:

• A comprehensive study on handcrafted feature design and selection process with machine learning models can achieve an AI-generated text detection success rate of up to 0.9993.

• A novel machine text detection method based on reverse engineering and text similarity analysis.

• An analysis of feature importance and explanation models to discuss our results on AI-generated detection.

II. RELATED WORKS

In this section, we summarized related research on AI- generated text detection methods.

A. Feature-Based Approach

In this approach, different NLP techniques are used to extract useful features from the text. The motivation for this approach is from the observations that NLG models create different artifacts in generated text [14]. Some most important features that have been proposed are frequencies, linguistics, fluency, and fact verification [14], [37], [38]. These features are then trained with machine learning models, such as SVM, RF, or neural networks, to build classification models [39]– [43].

B. Neural Language Model Approaches

For this approach, neural language models (NMLs) are built to detect generated text from state-of-the-art NLG models. Many NLG models, such as GPT or Grover, can be used to detect their own outputs. However, the performance is generally lower than a simple TF-IDF baseline model if no fine-tuning is performed. Therefore, fine-tuning pre-trained large language models is preferred in order to differentiate the human-written vs. NLG model output samples [14], [44], [45]. The disadvantages of this approach are a) the requirement for large datasets and b) heavy computation power and resources for training. This approach

has been applied in detecting machine-generated text in a specific domain rather than in a general context.

C. AI-Generated Text Detection on Specific Domains

Currently, there is no perfect AI-generated text detection in general (all possible domains). Much research focused on such detection tasks in specific domains, such as academic settings, scientific, fake news/reviews, or misinformation, as the scope can be narrowed down.

Academic settings: [21] collected submissions from computer science students and generated related ChatGPT re- sponges. They evaluated 8 LLM text detectors on the above data. Their research has provided insights into the detectors' performance for the educator to maintain academic integrity. [22] conducted similar research but covered 12 publicly avail- able AI-generated text tools and two commercial software (Turnitin and Plagiarism Check). Similarly, other research, such as [23]–[25] contributed additional useful insights into the importance of AI-generated text detectors in academic settings.

Scientific: [26] studied the gap between the scientific con- tent written by humans vs. generated by AI tools. The authors confirmed a "writing style" gap between them. [29] discussed the challenges of enforcing the policy on AI-generated pa- pers/journals. [27] recently proposed a text representation method with machine learning models to detect fake scientific abstracts generated by a GPT-3 based model. Similarly, [28] studies the performance of using plagiarism detectors and blinded human reviewers on differencing original abstracts vs. fake abstracts generated by ChatGPT.

Fake news/reviews or misinformation: [32]–[34] studied the impacts of using AI-generated texts that can cause missing- formation issues in the media. [36] conducted another study on the linguistic features and patterns of AI-generated COVID- 19 misinformation. On the other hand, [35] proposed an AI model to detect AI-generated fake restaurant reviews on social media. These studies have raised the alarm about the severity of abused AI tools in media communications and information diffusion.

III. PROPOSED METHODOLOGY

In this section, we discuss the proposed research method- ology 1. The methodology comprises two main sections: data collection and experiments. We provide a description of each section here.

A. Data Collection

To carry out our research, we require both synthetic and manually written text data for comparison purposes. As per our knowledge, there is no pre-existing database that provides this type of dataset. Therefore, our first step is to construct the dataset. In the second phase of our work, we will use NLP-based feature extraction to compare the features of the synthetic and human-generated data, train our model, and provide explanations.

We have generated two sets of data. The first one includes Wikipedia data and its corresponding artificially created data. The second one consists of news articles regarding the 2024 US election and their corresponding artificially created data.

1) Dataset Based on Wikipedia: We collected Wikipedia data using the method outlined in [46]. However, instead of only generating page summaries with ChatGPT, we expanded on this idea and created all sections of a Wikipedia page, which we labelled as synthetically generated data. The process for generating synthetic text for a Wikipedia page about dogs is shown in Figure 2. The page contains different sections, including Taxonomy, Evolution, Biology, and more. The text from these sections is extracted and libelled as human-written text (HWT). To generate text related to Taxonomy, we prompted ChatGPT with "Describe Dog Taxonomy." The text generated by ChatGPT is libelled as synthetically generated text (SGT). The same process was used for all other sections of the Wikipedia page to generate both HWT and SGT.

2) Dataset Based on News Articles: We gathered a dataset on the upcoming US Election in 2024 by collecting news articles and generating synthetic data 3. Initially, over 500 URLs of news articles related to the election were collected. Next, text data is extracted from each article and identified important keywords. Using these keywords, ChatGPT generated five questions for each article and prompted ChatGPT to answer those questions. As a result, we have generated five synthetic texts to correspond with each news article.

B. Feature Engineering

When working with datasets for NLP tasks, it is crucial to carry out feature engineering. This involves using similar techniques for both datasets to gain a better understanding of their patterns and characteristics. These features will be useful in classifying the datasets later in this research. We have hand-crafted basic NLP-related features, TD-IDF [47] related features, N-gram features [48], topic modelling features, readability score, named entity recognition (NER) [49] count, and text error length features. To find an optimal set of features, principal component analysis (PCA) [50] is used.

C. Classification

In order to differentiate between HWT and SGT, we utilized a dataset sourced from Wikipedia. We employed three different classifiers - Random Forest (RF) [51], Support Vector Machine (SVM) [52], and XGBoost (XGB) [53] - to perform the classification. Our models were trained, fine-tuned, and tested using this dataset. We used Precision, Recall, and F1 scores as metrics to evaluate and compare the performance of each model.

IV. RESULT AND DISCUSSION

In this section, data preprocessing and feature engineering will be described first. Then, the experiment setup, hyper- parameter tuning, and performance metrics are explained. Finally, the experiment results and discussion are presented.

A. Dataset Preprocessing and Feature Engineering

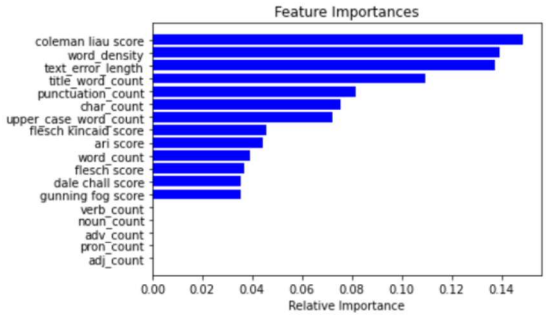
1) Data collection: As explained in section III-A, we col- lected two different kinds of raw datasets: - Wikipedia based dataset: For each section in the original Wikipedia articles, we have the human-written text as the ground truth (extracted from the original article) and the corresponding ChatGPT- generated text. - US election 2024 related news article dataset: For each article related to the US election 2024 event, we have the human-written text (extracted from the article) and a maximum of 10 related ChatGPT generated text (in the form of question answering based on the extracted keywords from the article).

2) Document Similarity: We utilized the Cosine similarity score to measure the similarity between HWT and SGT. The Cosine similarity score ranges from -1 to 1, where a score of 1 indicates that two documents are identical. If the score is less than 1, it means that the documents are not identical. As the score approaches -1, the degree of dissimilarity between the compared documents increases. In this research, we used the cosine similarity score to find the document similarity between HWT and SGT for the US Election related news articles based dataset. As mentioned in the III section, feature extraction is a prerequisite to calculating cosine similarity. Hence, we have calculated all features (as mentioned in Table I) before calculating the cosine similarity. Out of 50,783 features, most of them (35,742) belong to term frequencies, and they are sparse matrices. To observe the effect of the term frequencies, we have calculated two cosine similarities i) without term frequencies (15,041), and ii) with all features (50,783). In the first scenario, we are missing some information, and in the second scenario, we are using some features which are not needed. To solve both problems, we used PCA to identify the most useful features. Therefore, we used this new subset of features to find the cosine similarity score.

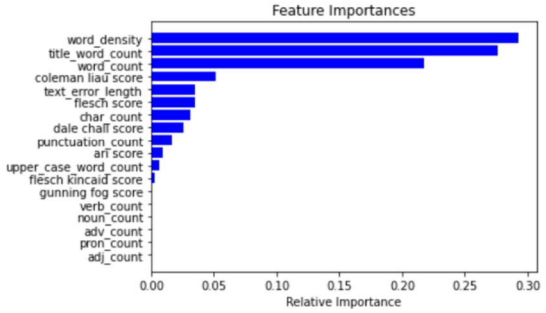
	Features without TF-IFD	
Cosinc scorc	Document count	Document %
-1 to -0.6	2	0.05%
-0.6 to -0.2	2787	76.50%
-0.2 to 0.6	817	22.43%
0.6 to 1	37	1.02%
	All Features	
Cosine score	Document count	Document %
-1 to -0.6	0	0.00%
-0.6 to -0.2	3537	97.09%
-0.2 to 0.6	106	2.91%
0.6 to 1	0	0.00%
	Features with PCA	
Cosine score	Document count	Document %
-1 to -0.6	0	0.00%
-0.6 to -0.2	3642	99.97%
-0.2 to 0.6	1	0.03%
0.6 to 1	0	0.00%

3) Feature Importance Analysis: For classification, the RF and XGB models performed similarly. To understand the behaviour of the training features, we have calculated the feature importance of both models using scikit-learn library [68]. Figure 4 shows the relative training feature importance graph. We can see that for the RF classifier, Coleman lieu score, word density, text error length, title word count, and punctuation count are the top five important training features; similarly, word density, title word count, word count,

Cole- man lieu score, text error length are the top five important features for XGB classifier. Out of the top five features, four features are common for both models. This gives a fair idea about the set of important features that can be used as the distinguishing characteristics between HWT and SGT. The Scikit-learn library has been used to calculate the im- portance of features in the entire training dataset. Additionally, the SHAP library [69] was utilized to explain the features for individual predictions. In figures 5a and 5b, the SHAP library has been used to display the individual prediction



(a) Feature Importance for Random Forest Classifier



(b) Feature Importance for XGB Classifier

V. CONCLUSION

In this paper, we proposed two approaches to detect AI- generated text: a) machine learning based and b) text similarity based methods. In order to test our proposed methodologies, we collected two different kinds of datasets: Wikipedia-based articles and US Election 2024 News articles. After that, we proposed four different groups of handcrafted features to be extracted from the raw text: Basic NLP features, Term Frequencies and Ngram features, Topic Modeling Features, and Other features (readability score, grammar error, and NER count). Then, we performed three experiments to demonstrate the effectiveness of our proposed methods. In the first experiment, after extracting the handcrafted features, we trained three machine learning models using RF, SVM, and XGBoost on the classification of human-written or ChatGPT-generated text. Both RF and XGBoost models perform well, with an F1 score of 0.9993. This approach proves to be effective in any domain with high accuracy in detection. To use this approach, we need to collect the ground truth texts (human-written) and ask AI tools such as ChatGPT to generate the artificial text based on the forming related ques- tions from the ground truth texts. Our designed handcrafted features can help develop effective machine-learning models for the detection of AI-generated texts.

The second experiment was designed to demonstrate our second approach to detecting AI-generated text. This approach does not require collecting ground truth texts. Given any text that we want to know, either human-written or AI- generated, based on topic modeling and keyword extraction, we formed related questions and asked AI tools to generate texts (ChatGPT in this project). Then, we applied a similar feature extraction in experiment 1. Finally, text similarity based on cosine will tell us the result. The results of our second experiment show that there is a clear classification boundary between human-written text and AI-generated text. This approach has proved effective and works in general situations, as no ground truth data needs to be collected.

REFERENCES

[1] T. Brants, A. C. Popat, P. Xu, F. J. Och, and J. Dean, "Large language models in machine translation," 2007.

[2] J. Wei, Y. Tay, R. Bommasani, C. Raffel, B. Zoph, S. Borgeaud, D. Yogatama, M. Bosma, D. Zhou, D. Metzler et al., "Emergent abilities of large language models," arXiv preprint arXiv:2206.07682, 2022.

[3] W. X. Zhao, K. Zhou, J. Li, T. Tang, X. Wang, Y. Hou, Y. Min, B. Zhang, J. Zhang, Z. Dong et al., "A survey of large language models," arXiv preprint arXiv:2303.18223, 2023.

[4] N. Indurkhya and F. J. Damerau, Handbook of Natural Language Processing, 2nd ed. Chapman & Hall/CRC, 2010.

[5] E. Reiter and R. Dale, Building Natural Language Generation Systems, ser. Natural Language Processing. Cambridge University Press, 2000. [Online]. Available: http://prp.contentdirections.com/mr/cupress. jsp/doi=10.2277/052102451X

[6] S. Bird, E. Klein, and E. Loper, Natural Language Processing with Python: Analyzing Text with the Natural Language Toolkit. Beijing: O'Reilly, 2009. [Online]. Available: http://www.nltk.org/book

[7] T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell et al., "Language mod- els are few-shot learners," Advances in neural information processing systems, vol. 33, pp. 1877–1901, 2020.

[8] "GPT-3.5 — lablab.ai," https://lablab.ai/tech/openai/gpt3-5, [Accessed 15-08-2023].

[9] H. Touvron, T. Lavril, G. Izacard, X. Martinet, M.-A. Lachaux, T. Lacroix, B. Rozi` ere, N. Goyal, E. Hambro, F. Azhar et al., "Llama: Open and efficient foundation language models," arXiv preprint arXiv:2302.13971, 2023.

[10] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "Bert: Pre-training of deep bidirectional transformers for language understanding," arXiv preprint arXiv:1810.04805, 2018.